

Image Processing - Lecture 2



Amir Atapour-Abarghouei

amir.Atapour-abarghouei@durham.ac.uk

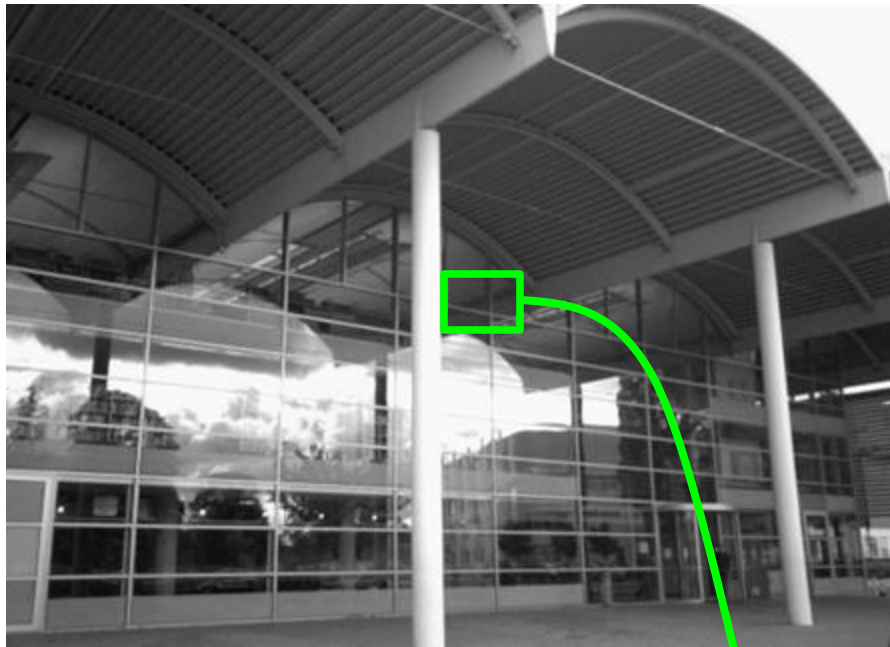
Reading:

Szeliski – Section 3.1.1 / 3.1.3 (related)

Gonzalez/Woods – Section 2.6 + 3.2.1

Point Transforms – Arithmetic and Logical Operations

Recap ... Image Representation



- Images are 2D signals.
- Signals can be digitised as a structured set of numbers.
- Images can be represented by 2D arrays or matrices.

138	136	67	111	152	82	84	150	106	69	136	131	65	111	150	77	81	152	104
147	125	64	123	148	72	94	155	96	72	146	121	63	124	141	67	95	151	86
79	88	153	104	70	139	130	65	115	150	76	85	152	101	65	137	127	60	114
66	108	154	81	81	150	108	65	136	135	63	109	152	77	78	150	104	60	134
88	79	149	112	67	131	135	66	106	152	80	77	150	105	62	132	131	62	105
91	75	149	116	63	127	140	68	101	154	86	75	148	113	62	128	137	63	99
149	117	63	127	144	68	99	155	89	74	147	115	62	124	138	67	97	150	86
149	121	61	124	144	69	96	154	91	72	145	118	62	120	142	71	93	149	90
140	129	64	114	148	76	86	154	101	66	139	126	62	115	145	71	89	151	94
139	133	64	114	150	75	85	152	101	65	138	126	63	114	145	72	86	151	96

`image[x][y] = 150`

If The Image Is a Matrix ...

... then, we can process (**transform**) an image by manipulating the corresponding matrix.



A



$0.33 \cdot A$

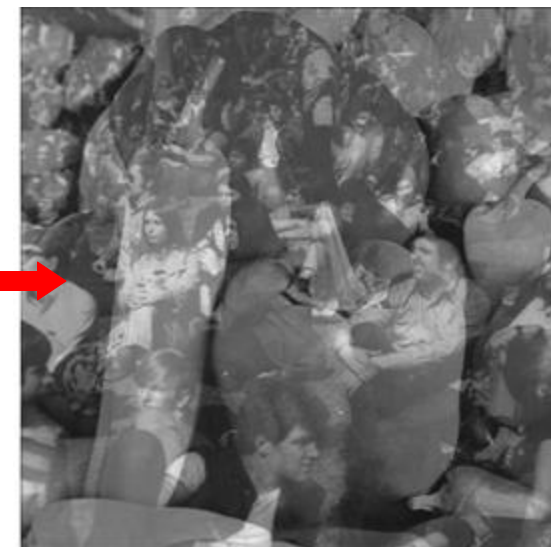
We can also process (**transform**) multiple images by manipulating the corresponding matrices.



A



B



$$0.5 \cdot A + 0.5 \cdot B$$

Element-wise

We can process (**transform**) each pixel, p , individually.

$$f(p) = p/3$$



These processing methods **transform** the image from one state to another.

Why Do Transformations?

- Remove degradations introduced during capture
- Improve image appearance for viewing or further processing (i.e. image enhancement)
- Identify image features for recognition of scene objects (i.e. image analysis / computer vision)
- Transform image to alternative representation (**domain transform**) for better processing (e.g. Fourier transform - later in the course).

Types of image transforms

An image transform, processing I_{input} to I_{output} , may be:

- a **point transform** involving only a single pixel at a time
- a **local transform** involving the local image neighbourhood (pixel + those immediately “next to it” - later in the course)
- a **global transform** involving the whole image (later in the course).

Basic Point Transforms

- Point transforms map individual points in the input image to individual points in the output image.
- Performed as an operation, denoted $\circ$, between two images, I_A and I_B , **or** between and image, I_A , and a constant value, C :

$$I_O = I_A \circ I_B$$

$$I_O = I_A \circ C$$

Since it is element-wise, there must be correspondence, i.e. images must have the same dimensions!



Basic Point Transforms

- Pixel location (i,j) in the output image is computed as follows:

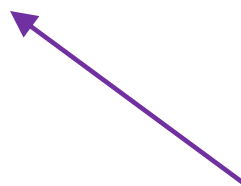
$$I_o(i,j) = I_A(i,j) \circ I_B(i,j)$$

$$I_o(i,j) = I_A(i,j) \circ C$$

- Iterate over the image indices for $(i,j) = \{0 \dots w-1, 0 \dots h-1\}$.

w = image width

h = image height

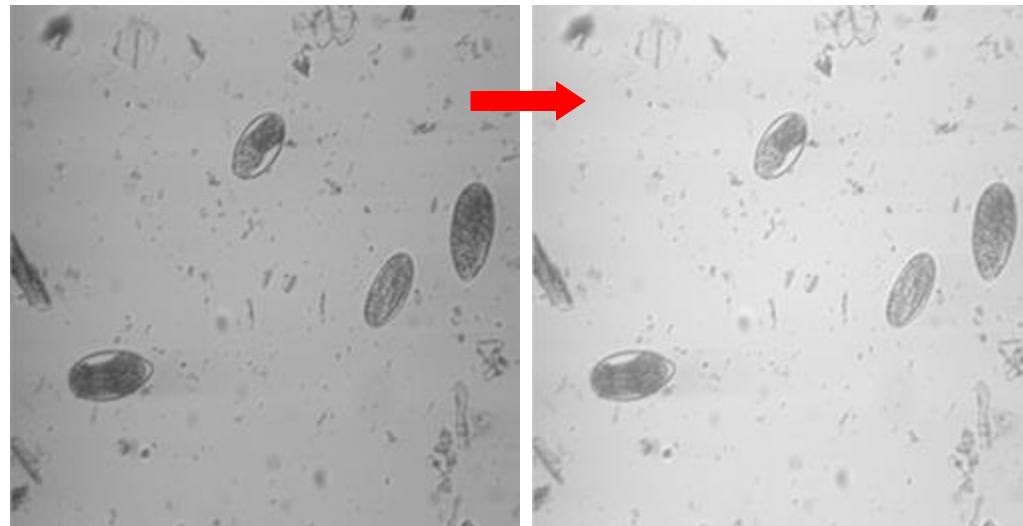


Think of a double loop, though modern languages have more efficient code for this

Today - simple arithmetic and logical transforms
which can be highly effective

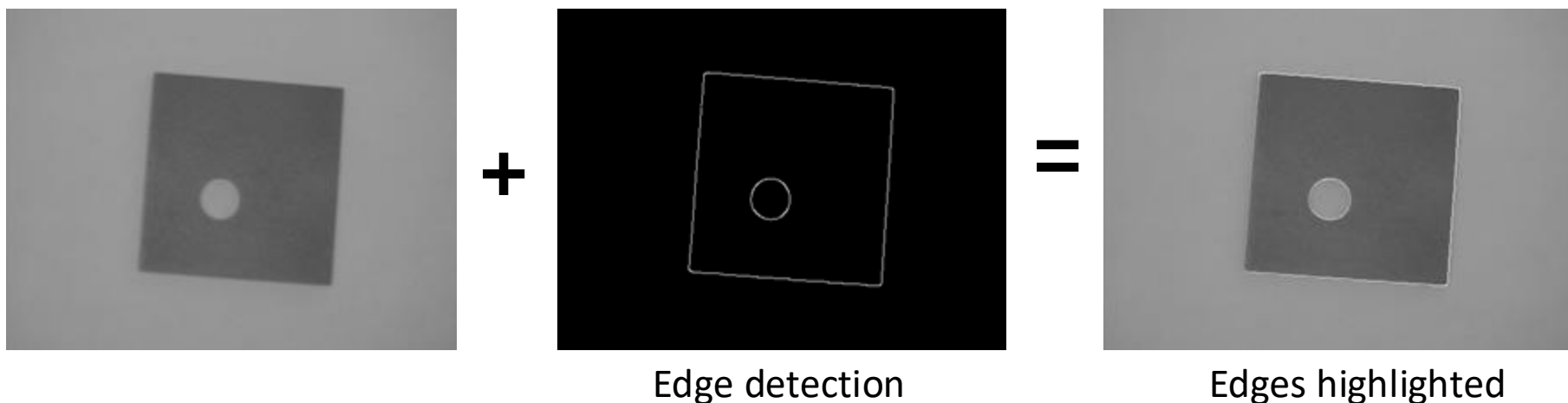
Arithmetic Operations: Addition

- **Operation:** adding a value to each image pixel.
- **Application:** brightness adjustment - adding a positive constant value to each pixel increases brightness. (beware of clipped values)



Arithmetic Operations: Addition

- **Application:** **blending** - adding images together produces a composite image of both inputs.

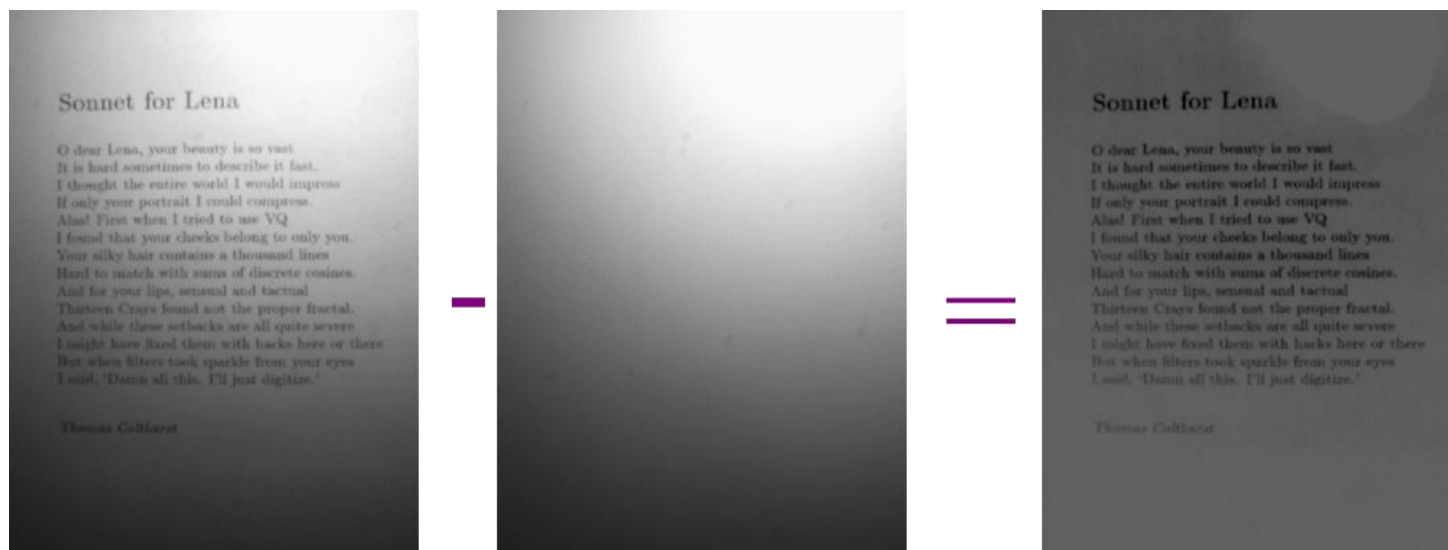


In both cases watch for integer overflow. The result of the addition could be out of range.

Arithmetic Operations: Subtraction

- **Operation:** subtracting a value from each image pixel.
- **Application:** brightness adjustment - as per addition.

Image differencing - subtracting one image from another shows us the difference between images.



- Useful Variation:

$$I_o = |I_A - I_B|$$

(e.g. correction of non-uniform illumination in document scanner – just subtract a “blank scan”)

Arithmetic Operations: Subtraction



Arithmetic Operations: Subtraction



Arithmetic Operations: Subtraction



Arithmetic Operations: Subtraction



What has moved in the scene and where?

Arithmetic Operations: Subtraction



Naïve image differencing suffers from noise.

Arithmetic Operations: Subtraction

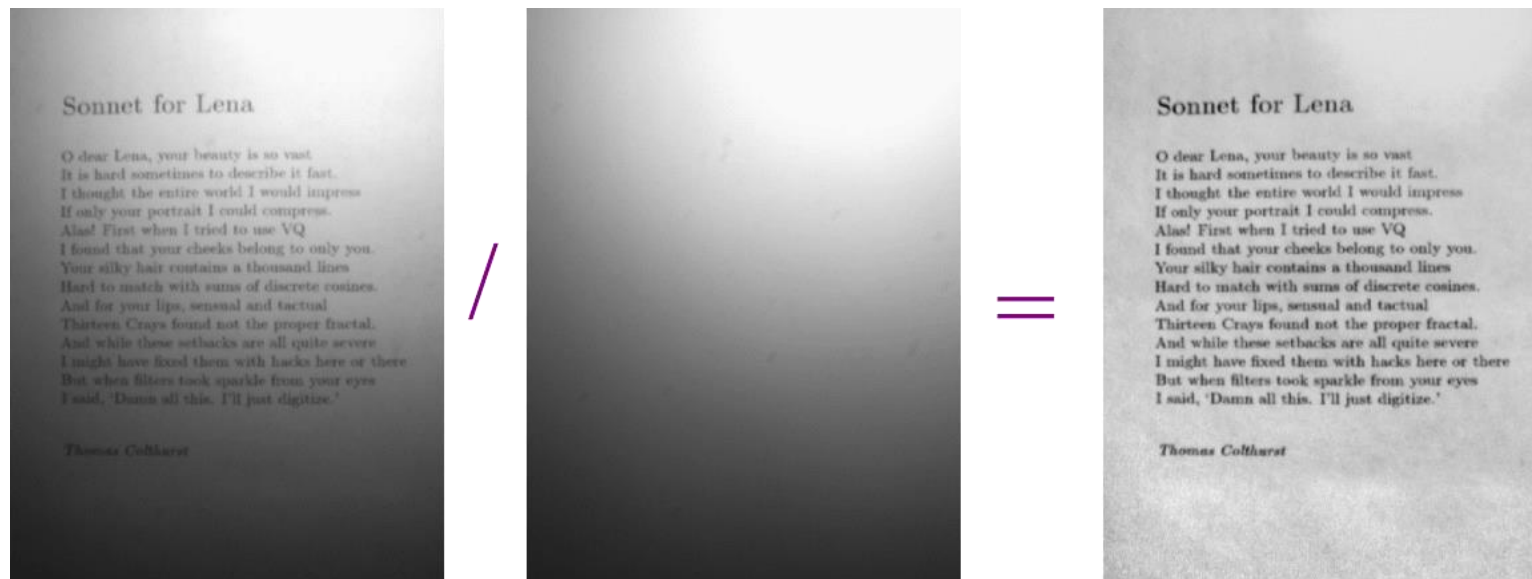


Is the change
significant or
is it noise?

Absolute differencing with some intelligent **thresholding and processing** added → the basics of an intelligent surveillance system.

Arithmetic Operations: Division

- **Operation:** dividing each image pixel by a value.
- **Application:** brightness adjustment - scale image intensities, e.g. reduce to 25% of the original by dividing by 4 or:



- Operation results small and floating point (need to normalise/quantise).

Arithmetic Operations: Division

- **Application:** **image differencing** - dividing an image by another.
(beware of division by zero, usually add a constant)
 - **result == 1** where the pixel values are identical and
result != 1 where differences occur.
 - Image differencing via subtraction is more efficient.



Arithmetic Operations: Multiplication

- **Operation:** multiplying each image pixel by a value (equivalent to division by the inverse of that value).
- **Application:** brightness adjustment - pixel value scaling as per division.



HIPR2

x3
→



Application: Image Blending

- Image blending is an application of image arithmetic operations producing ghosting or overlay effects between different images.
- N images $I_1, I_2, \dots, I_N$ can be blended in equal proportions as:

$$I_{output} = \sum_i \frac{1}{N} I_i$$

- Alternatively, different weights can be used between images to enhance/suppress the features of different images in the final result.

Application: Image Blending



Blending images

Logical Operations: **NOT** (inverse)

- **Operation:** logical NOT to invert the image.
 - For **binary images**, white pixels become black and vice versa.
 - For 8-bit greyscale images (or 8-bit colour channels):

$$I_{output} = 255 - I_{input}$$

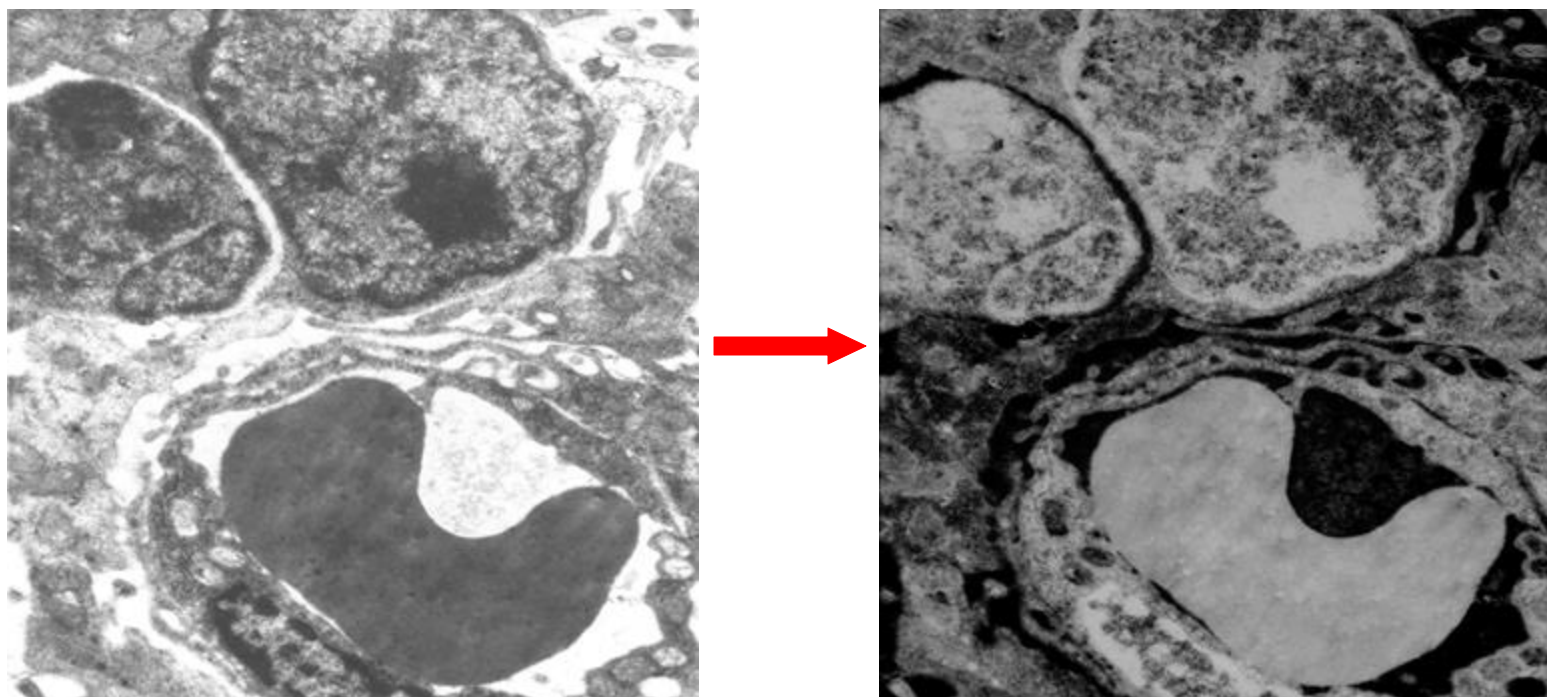
Logical Operations: **NOT** (inverse)

- In greyscale images, dark areas become light and vice versa.
- In colour (and greyscale) images, it produces the **photographic negative effect** (not negative values!).



Logical Operations: **NOT** (inverse)

When applied to grayscale images, it highlights different features/structures.



- Visualisation of a medical slide.
- If an image is dark overall, its inverse might enhance features.

Logical Operations: **AND**

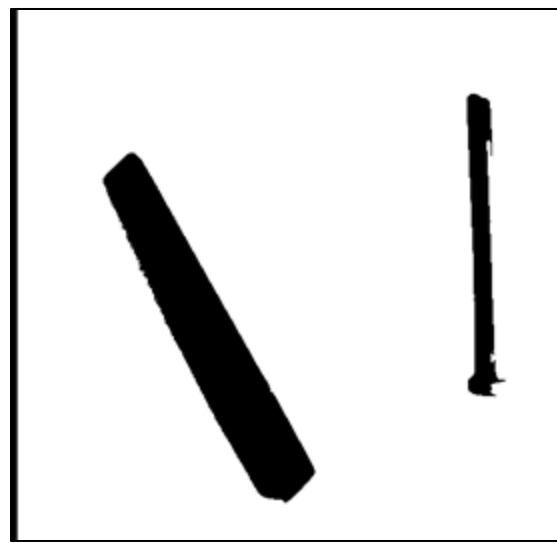
- **Operation:** logical AND.
- **Application:**
 - detecting differences or overlap between images
 - highlighting appropriate regions with a mask
 - slicing bit planes through an image.



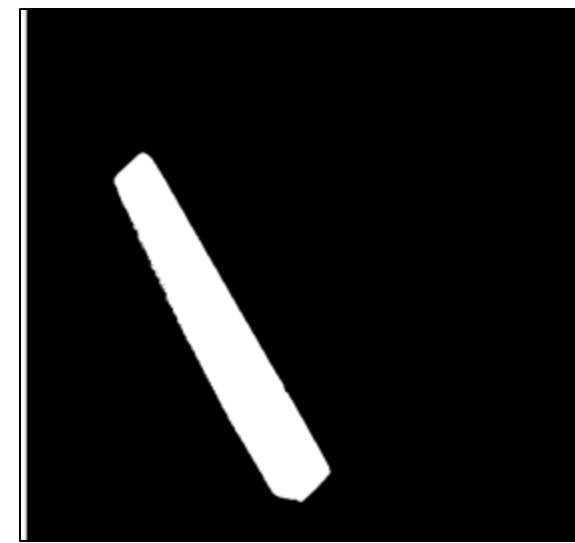
Logical Operations: **AND**



A



B

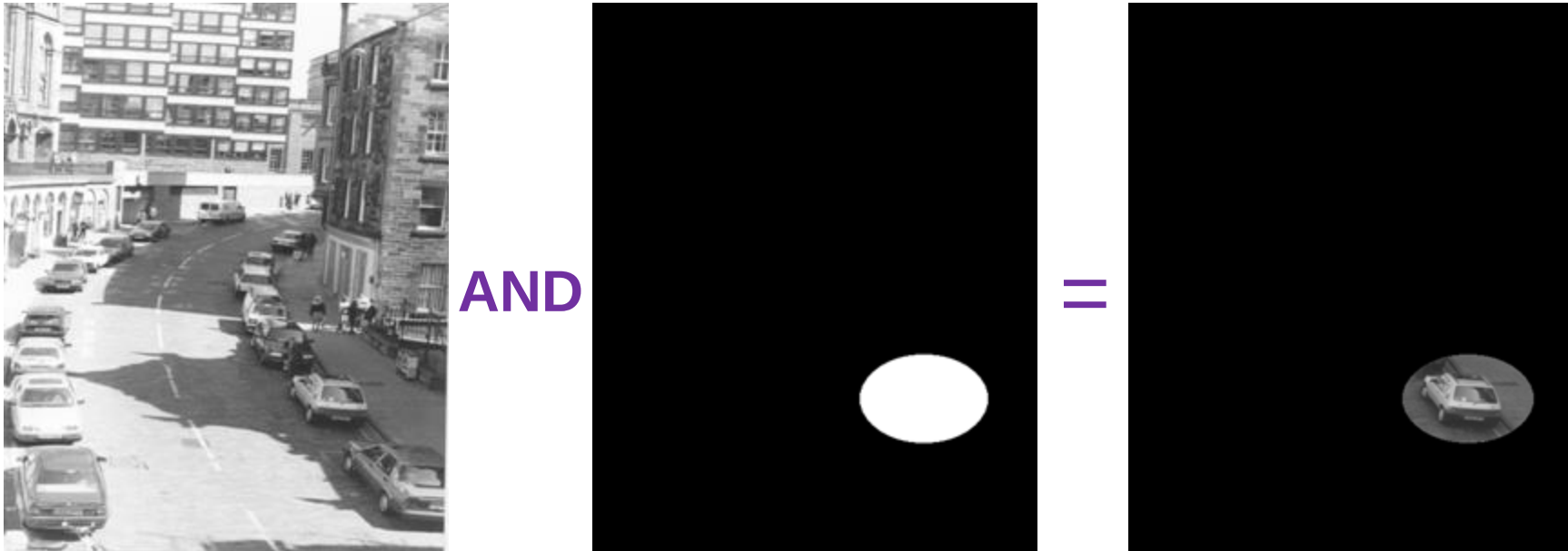


NOT(A) AND NOT(B)

ANDing the inverse of two binary images.

Object that did not move is in the most significant bit.

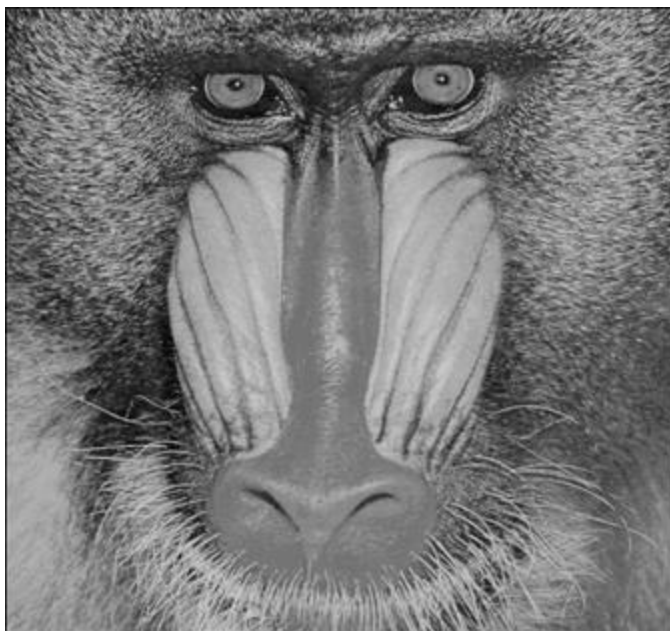
Logical Operations: **AND**



- **Application:** selecting an individual item or region.

Logical Operations: **AND**

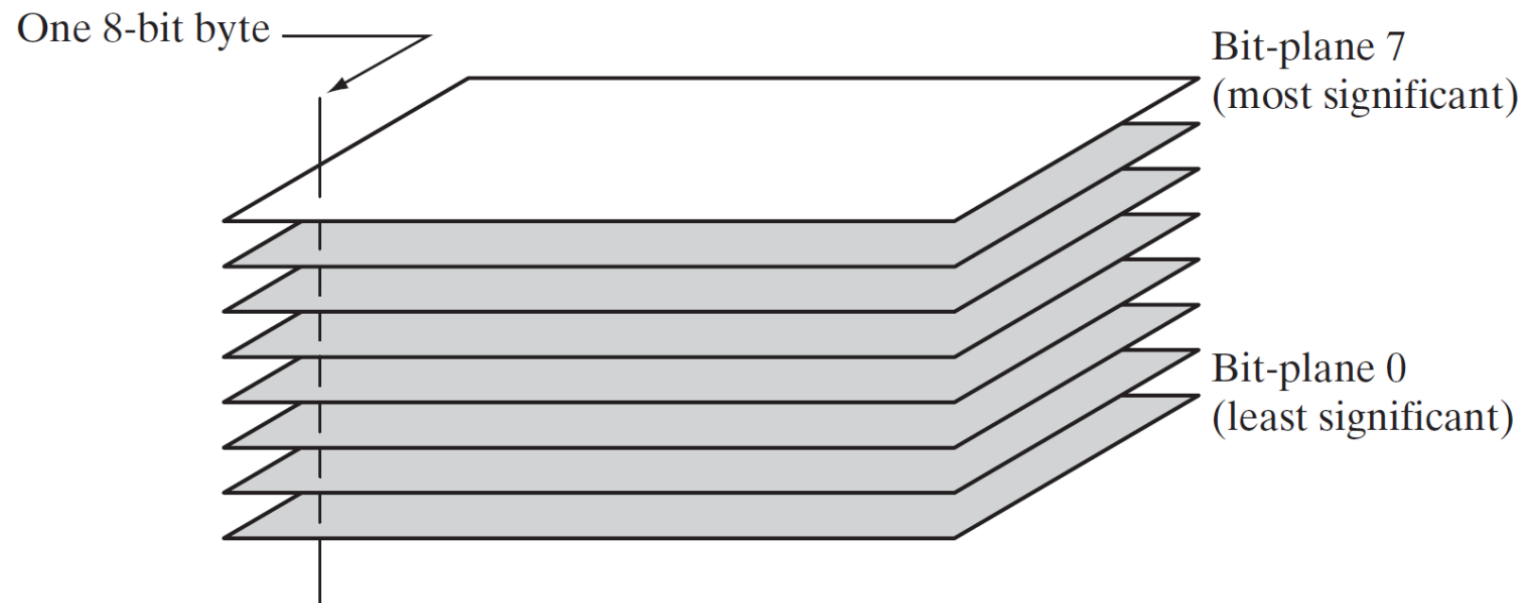
- **Application:** Bit plane slicing by logical AND.
- For example, bitwise ANDing with 128 (10000000 binary) gives the most significant bit plane.



7th, 6th and 4th bit

Bit Planes

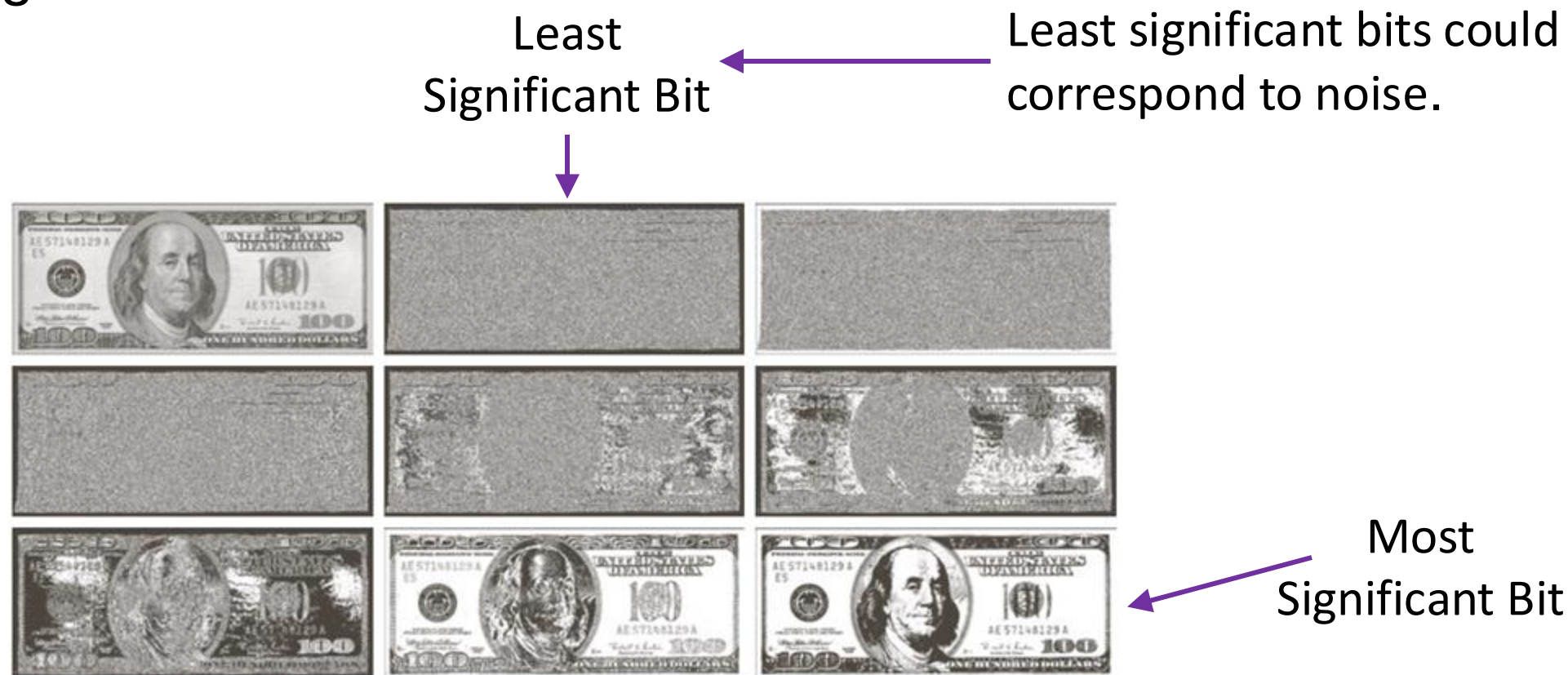
- Pixel values are integers composed of bits.
- 8 bits (1 byte) in a 256-level greyscale image.
- **Logical** (bit-wise) **operations** can be thought of as applied to **bit planes**.



Bit plane representation of an 8-bit image.

Bit Planes

Level of detail (or noise/texture) associate to each bit of each pixel in a greyscale image:



Logical Operations: OR

- **Operation:** Logical OR.
 - useful for processing binary images (0 or 1) images.
 - also to detect common or moved objects.

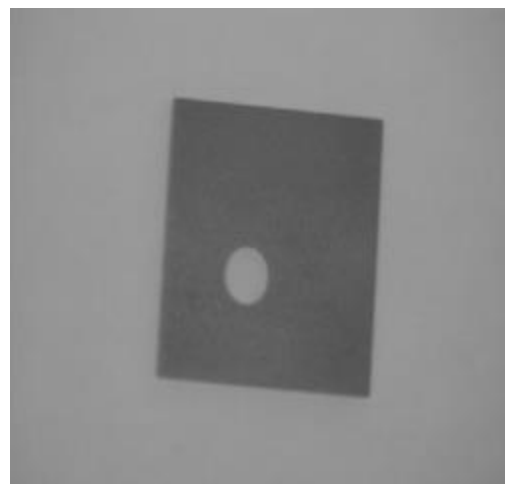


From Boolean
Algebra,
De Morgan's Laws

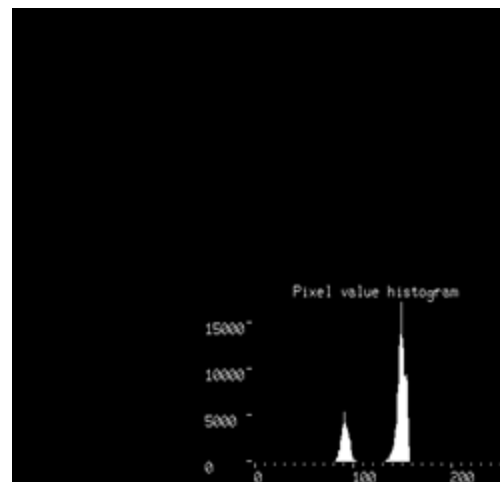
An OR operation can be replaced by AND and NOT operations via:

$$A \text{ OR } B = \text{NOT}(\text{NOT}(A) \text{ AND } \text{NOT}(B))$$

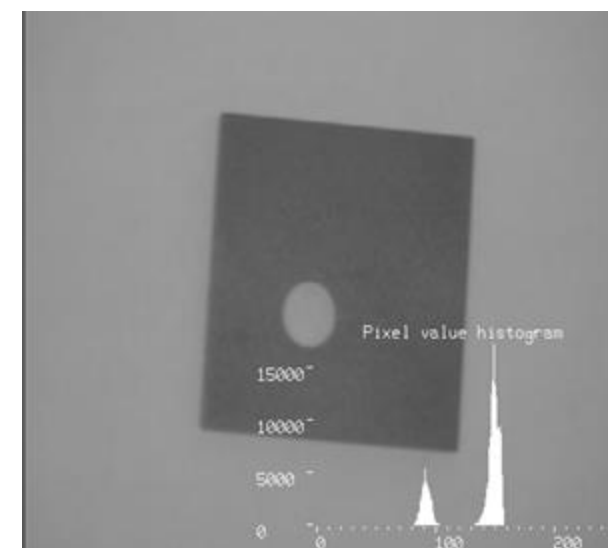
Logical Operations: **OR**



OR



=



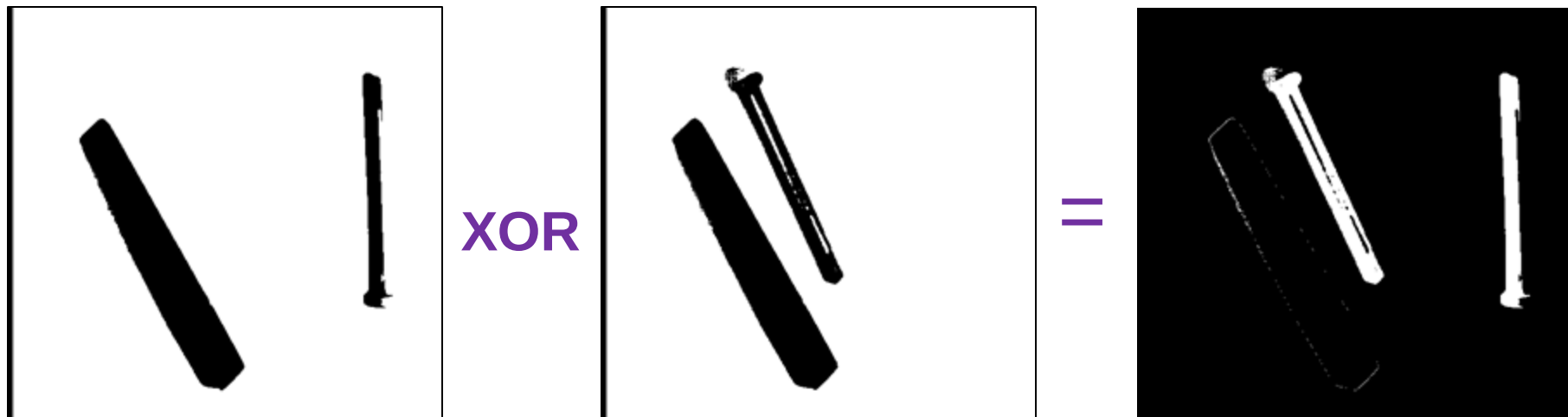
Can do basic image overlays,
but no control on weighting available.

Logical Operations: XOR

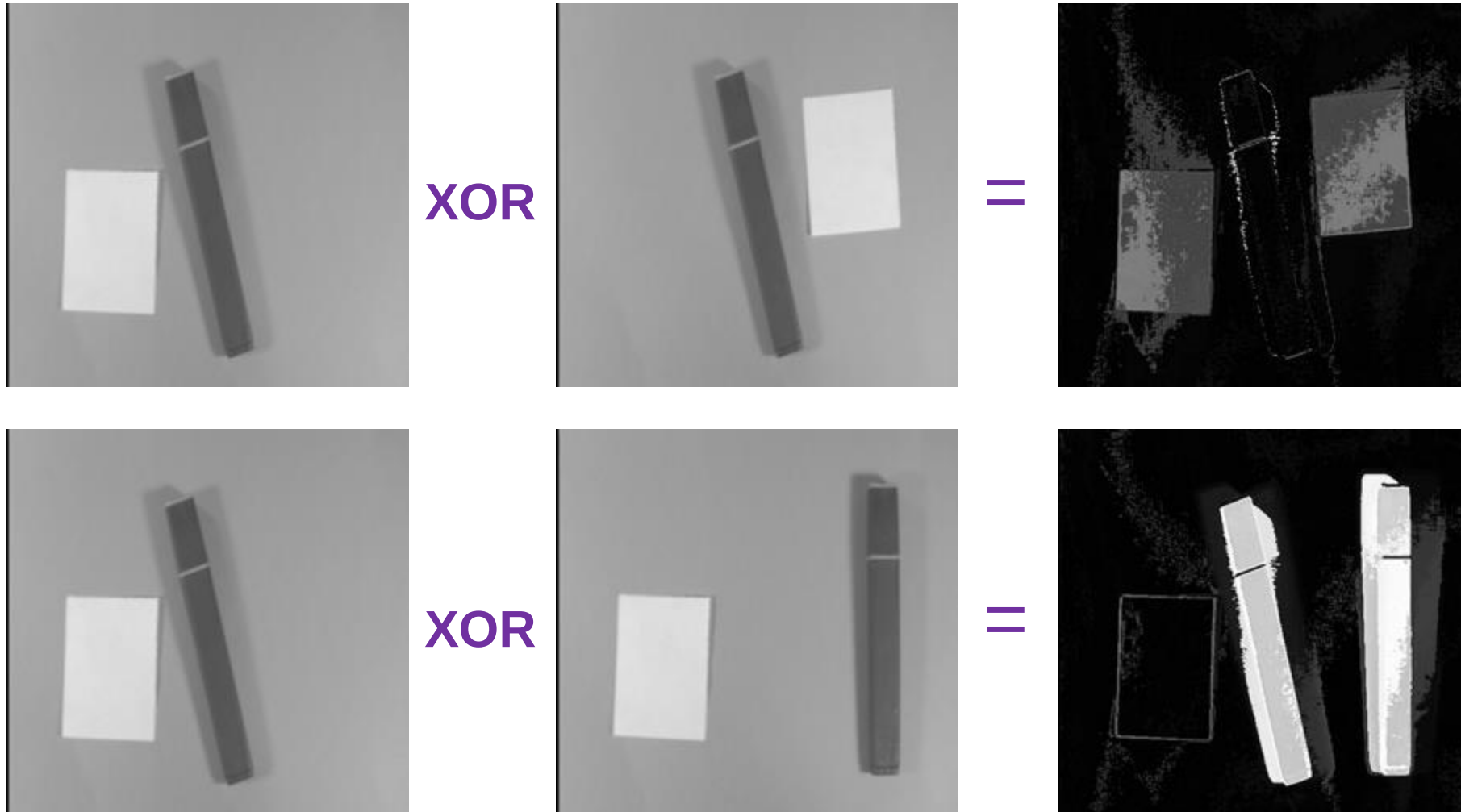
- **Operation:** logical XOR.
 - Useful in efficiently detecting image differences.
 - Highlights only where changes occur!

Truth table

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0



Logical Operations: XOR



**Also their inversions
NOR, NXOR and NAND**

**(all are combinations of what we
have already seen)**

Colour to Greyscale



Colour



Greyscale

How do we transform to grayscale?

Colour to Greyscale

- Initial step in many algorithms
 - Greyscale contains less information.
 - It is a single-channel image.
 - It is easier/faster to process.
 - It is a non-invertible, lossy transform (information is destroyed).
- Important feature-related information is maintained:
 - edges, regions, blobs, junctions.



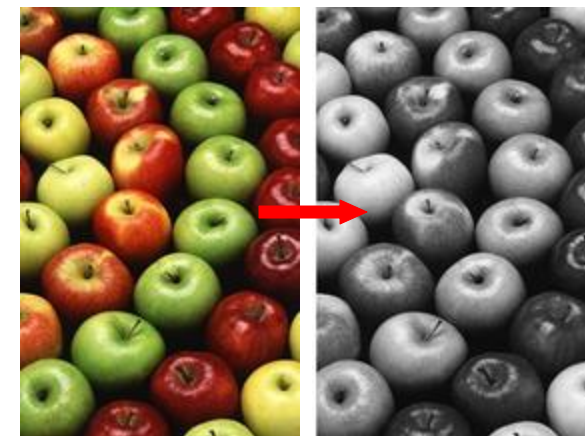
Colour to Greyscale

- A simple way to do the conversion is to take a weighted sum of the R,G,B values:
 - input RGB colour image, I_c
 - output Grayscale image, I_g

$$I_g = \alpha I_c(R) + \beta I_c(G) + \gamma I_c(B)$$

- The coefficients (α , β , γ):
 - in **proportion to human vision sensitivity** to the R,G,B channels
 - One commonly used weighting for the conversion is (NTSC television standard, derived from psychophysical experiments):

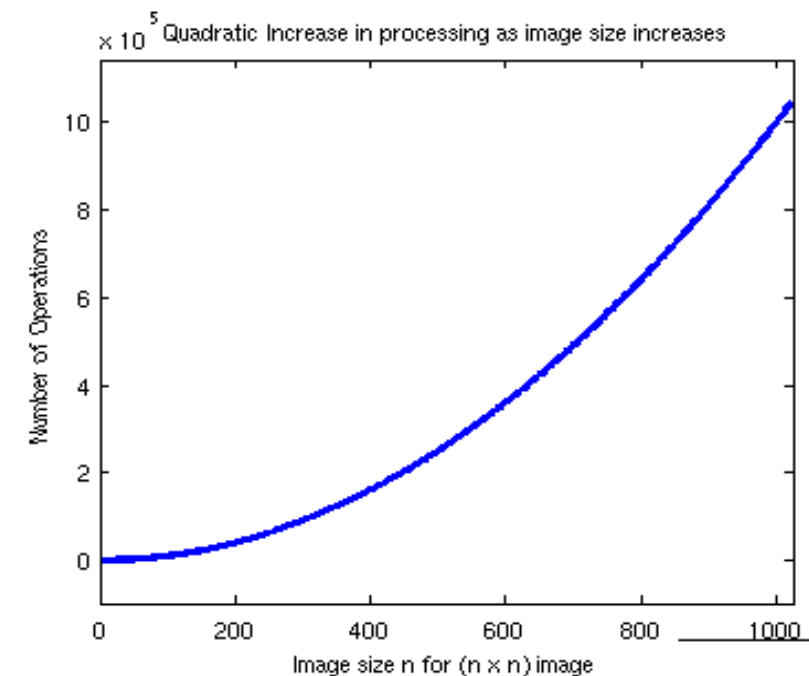
$$\alpha = 0.2989, \quad \beta = 0.5870 \quad \text{and} \quad \gamma = 0.1140$$



**One final thing to think about
when processing images ...**

Computational Complexity

- Square $n \times n$ images have n^2 pixels.
- Computational cost of a point transform applied to the whole image = $O(n^2)$.
- **Quadratic** runtime means that the total number of CPU operations increases rapidly as the image size increases.
- Solutions:
 - **parallel processing** (traditional)
 - **image pyramid approaches**: down-size + processing + up-size again.



Quick Demonstration



- Input: image or feed from the webcam
- Output: *Grayscale, Grayscale / 2, abs. diff. with previous frame*

https://github.com/atapour/ip-python-opencv/blob/main/arithmetic_point_transforms.py



- Input: image or feed from the webcam
- Output: *NOT, AND with a mask, XOR with previous frame*

https://github.com/atapour/ip-python-opencv/blob/main/logical_point_transforms.py